

Tugas Praktikum

Tugas 1: Kalkulator Paralel Buat program yang:

1. Menerima input 2 angka dari user
2. Menghitung penjumlahan, pengurangan, perkalian, dan pembagian
3. Setiap operasi dikerjakan oleh thread berbeda
4. Tampilkan semua hasil setelah semua thread selesai

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
// Variabel global
```

```
double a, b;
```

```
double hasil_jumlah, hasil_kurang, hasil_kali, hasil_bagi;
```

```
// Fungsi thread
```

```
void* tambah(void* arg) {  
    hasil_jumlah = a + b;  
    pthread_exit(NULL);  
}
```

```
void* kurang(void* arg) {  
    hasil_kurang = a - b;  
    pthread_exit(NULL);  
}
```

```
void* kali(void* arg) {  
    hasil_kali = a * b;  
    pthread_exit(NULL);  
}
```

```

void* bagi(void* arg) {
    if (b != 0)
        hasil_bagi = a / b;
    else
        hasil_bagi = 0; // hindari pembagian nol
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2, t3, t4;

    printf("Masukkan angka pertama: ");
    scanf("%lf", &a);
    printf("Masukkan angka kedua: ");
    scanf("%lf", &b);

    // Membuat 4 thread
    pthread_create(&t1, NULL, tambah, NULL);
    pthread_create(&t2, NULL, kurang, NULL);
    pthread_create(&t3, NULL, kali, NULL);
    pthread_create(&t4, NULL, bagi, NULL);

    // Menunggu semua thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);
}

```

```
// Tampilkan hasil

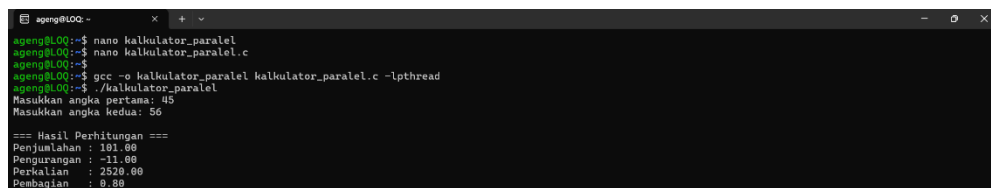
printf("\n=== Hasil Perhitungan ===\n");
printf("Penjumlahan : %.2lf\n", hasil_jumlah);
printf("Pengurangan : %.2lf\n", hasil_kurang);
printf("Perkalian   : %.2lf\n", hasil_kali);
if (b != 0)
    printf("Pembagian   : %.2lf\n", hasil_bagi);
else
    printf("Pembagian   : Error (pembagian nol)\n");

return 0;
}
```

2. Compile dan jalankan

```
gcc -o kalkulator_paralel kalkulator_paralel.c -lpthread
./kalkulator_paralel
```

3. Dokumentasi



```
ageng@140: ~$ nano kalkulator_paralel
ageng@140: ~$ nano kalkulator_paralel.c
ageng@140: ~$ gcc -o kalkulator_paralel kalkulator_paralel.c -lpthread
ageng@140: ~$ ./kalkulator_paralel
Masukkan angka pertama: 45
Masukkan angka kedua: 56
=== Hasil Perhitungan ===
Penjumlahan : 101.00
Pengurangan : -11.00
Perkalian   : 2520.00
Pembagian   : 0.80
```

Tugas 2:

File Processing Buat program yang:

1. Membaca file teks
2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
3. Gabungkan hasilnya
4. Bandingkan waktu eksekusi dengan versi single-thread

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <pthread.h>
```

```
#include <time.h>
```

```
typedef struct {
```

```
    char *data;
```

```
    long start;
```

```
    long end;
```

```
    int count;
```

```
} ThreadArg;
```

```
// Fungsi untuk menghitung kata dalam sebagian teks
```

```
void* hitung_kata(void* arg) {
```

```
    ThreadArg* t = (ThreadArg*) arg;
```

```
    int in_word = 0;
```

```
    int count = 0;
```

```
    for (long i = t->start; i < t->end; i++) {
```

```
        if (t->data[i] == ' ' || t->data[i] == '\n' || t->data[i] == '\t') {
```

```
            in_word = 0;
```

```

        } else if (!in_word) {
            in_word = 1;
            count++;
        }
    }

    t->count = count;
    pthread_exit(NULL);
}

// Fungsi menghitung kata single-thread
int hitung_kata_single(const char *data, long size) {
    int in_word = 0, count = 0;
    for (long i = 0; i < size; i++) {
        if (data[i] == ' ' || data[i] == '\n' || data[i] == '\t') {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

int main() {
    char filename[100];
    printf("Masukkan nama file teks: ");
    scanf("%s", filename);

```

```

// Baca file
FILE *fp = fopen(filename, "r");
if (!fp) {
    perror("Gagal membuka file");
    return 1;
}

fseek(fp, 0, SEEK_END);
long size = ftell(fp);
rewind(fp);

char *data = malloc(size + 1);
fread(data, 1, size, fp);
data[size] = '\0';
fclose(fp);

// === Versi Single Thread ===
clock_t start_single = clock();
int total_single = hitung_kata_single(data, size);
clock_t end_single = clock();

// === Versi Multi Thread (2 bagian) ===
long mid = size / 2;

// Pastikan pembagian tidak memotong kata di tengah
while (mid < size && data[mid] != ' ' && data[mid] != '\n' && data[mid] != '\t')
    mid++;

ThreadArg arg1 = { data, 0, mid, 0 };

```

```

ThreadArg arg2 = { data, mid, size, 0 };

pthread_t t1, t2;
clock_t start_multi = clock();

pthread_create(&t1, NULL, hitung_kata, &arg1);
pthread_create(&t2, NULL, hitung_kata, &arg2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

int total_multi = arg1.count + arg2.count;
clock_t end_multi = clock();

// === Hasil ===
printf("\n=== HASIL PERHITUNGAN ===\n");
printf("Single-thread: %d kata\n", total_single);
printf("Multi-thread : %d kata\n", total_multi);

double waktu_single = (double)(end_single - start_single) /
CLOCKS_PER_SEC;
double waktu_multi = (double)(end_multi - start_multi) /
CLOCKS_PER_SEC;

printf("\nWaktu eksekusi single-thread : %.6f detik\n", waktu_single);
printf("Waktu eksekusi multi-thread : %.6f detik\n", waktu_multi);

free(data);
return 0;
}

```

2. Compile dan jalankan

```
gcc -o file_processing file_processing.c -lpthread
```

```
./file_processing
```

3. Dokumentasi

```
ageng@L00:~$ nano file_processing.c
ageng@L00:~$ gcc -o file_processing file_processing.c -lpthread
ageng@L00:~$ nano contoh.txt
ageng@L00:~$ ./file_processing
Masukkan nama file teks: contoh.txt

=== HASIL PERHITUNGAN ===
Single-thread: 425 kata
Multi-thread : 425 kata

Waktu eksekusi single-thread : 0.000022 detik
Waktu eksekusi multi-thread : 0.000021 detik
ageng@L00:~$ nano producer_consumer.c
```


Tugas 3: Producer-Consumer

Buat simulasi producer-consumer dengan:

1. Thread producer yang menghasilkan angka random
2. Thread consumer yang memproses angka tersebut
3. Gunakan mutex untuk sinkronisasi
4. Buffer maksimal 10 item

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define BUFFER_SIZE 10
```

```
int buffer[BUFFER_SIZE];
```

```
int count = 0; // jumlah item dalam buffer
```

```
pthread_mutex_t mutex;
```

```
pthread_cond_t cond_full;
```

```
pthread_cond_t cond_empty;
```

```
// Fungsi Producer
```

```
void* producer(void* arg) {
```

```
    while (1) {
```

```
        int item = rand() % 100; // angka random
```

```
        pthread_mutex_lock(&mutex);
```

```
        // Jika buffer penuh → tunggu
```

```
        while (count == BUFFER_SIZE) {
```

```
            pthread_cond_wait(&cond_full, &mutex);
```

```

    }

    // Tambah item ke buffer
    buffer[count++] = item;
    printf("Producer menghasilkan: %d (total: %d)\n", item, count);

    pthread_cond_signal(&cond_empty); // beri tahu consumer
    pthread_mutex_unlock(&mutex);

    sleep(1); // jeda agar terlihat prosesnya
}
return NULL;
}

// Fungsi Consumer
void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        // Jika buffer kosong → tunggu
        while (count == 0) {
            pthread_cond_wait(&cond_empty, &mutex);
        }

        // Ambil item dari buffer
        int item = buffer[--count];
        printf("Consumer memproses : %d (tersisa: %d)\n", item, count);

        pthread_cond_signal(&cond_full); // beri tahu producer
        pthread_mutex_unlock(&mutex);
    }
}

```

```
        sleep(2); // waktu proses lebih lama dari produksi
    }
    return NULL;
}
```

```
int main() {
    pthread_t prod, cons;

    srand(time(NULL));
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond_full, NULL);
    pthread_cond_init(&cond_empty, NULL);

    // Buat thread producer dan consumer
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // Tunggu kedua thread (program jalan terus)
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond_full);
    pthread_cond_destroy(&cond_empty);

    return 0;
}
```

2. Compile dan jalankan

```
gcc -o producer_consumer producer_consumer.c -lpthread
```

```
./producer_consumer
```

3. Dokumentasi

```
ageng@LQ:~$ nano producer_consumer.c
ageng@LQ:~$ gcc -o producer_consumer producer_consumer.c -lpthread
ageng@LQ:~$ ./producer_consumer
Producer menghasilkan: 17 (total: 1)
Consumer memproses : 17 (tersisa: 0)
Producer menghasilkan: 59 (total: 1)
Producer menghasilkan: 94 (total: 2)
Consumer memproses : 94 (tersisa: 1)
Producer menghasilkan: 77 (total: 2)
Producer menghasilkan: 53 (total: 3)
Consumer memproses : 53 (tersisa: 2)
Producer menghasilkan: 18 (total: 3)
Producer menghasilkan: 48 (total: 4)
Consumer memproses : 48 (tersisa: 3)
```